SchemaRAG: A Schema-aware Retrieval-Augmented Generation Framework for Text-to-SQL

DI WU, Southwest University, China
ZETONG TANG, Southwest University, China
YI HE, College of William & Mary, United States
XIN LUO*, Southwest University, China

Text-to-SQL refers to the task of converting natural language queries into Structured Query Language (SQL), enabling users to interact with databases without knowing SQL. Large Language Models (LLMs) have demonstrated considerable potential in implementing Text-to-SQL through retrieval-augmented generation and prompt engineering. However, these methods still face challenges in effectively understanding complex database schemas, failing to produce valid SQL queries. To address this issue, this paper proposes a Schema-aware Retrieval-Augmented Generation (SchemaRAG) framework with three core components. First, a SchemaLinker is fine-tuned to align natural language with schema items by knowledge distilling from high-quality chain-of-thought data, where its reasoning capabilities are further refined through group relative policy optimization. Second, a schema-augmented retriever is designed to retrieve the most relevant examples by referencing the database schema, thereby enhancing the LLM's ability to understand and generate SQL syntax. Finally, SchemaRAG adopts a Pareto-optimal selection mechanism to identify the final SQL query from a set of high-quality candidates to enhance robustness. As such, SchemaRAG can effectively learn complex database schemas to syntactically align with the structures of SQL, thereby generating more valid SQL queries. Extensive experiments on five benchmark datasets are conducted across several mainstream LLMs. The results demonstrate that SchemaRAG significantly outperforms four state-of-the-art Text-to-SQL competitors. The source code, datasets, and appendix of this paper are available at **https://github.com/chelsea2002/SchemaRAG**.

CCS Concepts: • **Information systems** → **Structured Query Language**.

Additional Key Words and Phrases: Text-to-SQL, Semantic Alignment, Retrieval-Augmented Generation, Large Language Model

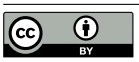
ACM Reference Format:
Di Wu, Zetong Tang, Yi He, and Xin Luo. 2026. SchemaRAG: A Schema-aware Retrieval-Augmented Generation Framework for Text-to-SQL. *Proc. ACM Manag. Data* 4, 1 (SIGMOD), Article 82 (February 2026), 26 pages. https://doi.org/10.1145/3786696

1 Introduction

Text-to-SQL aims to automatically translate natural language questions into executable Structured Query Language (SQL) queries, thereby lowering the technical barrier for non-expert users to perform data operations. Although methods based on Large Language Models (LLMs) have achieved significant progress, they still face a critical challenge: the massive parameter scale of LLMs can introduce content hallucination[16, 43], which compromises the accurate interpretation of user

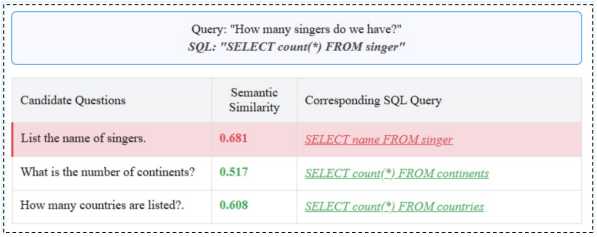
*Corresponding author

Authors' Contact Information: Di Wu, Southwest University, Chongqing, China, wudi1986@swu.edu.cn; Zetong Tang, Southwest University, Chongqing, China, xndx029311@email.swu.edu.cn; Yi He, College of William & Mary, Williamsburg, Virginia, United States, yihe@wm.edu; Xin Luo, Southwest University, Chongqing, China, luoxin@swu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2836-6573/2026/2-ART82
https://doi.org/10.1145/3786696

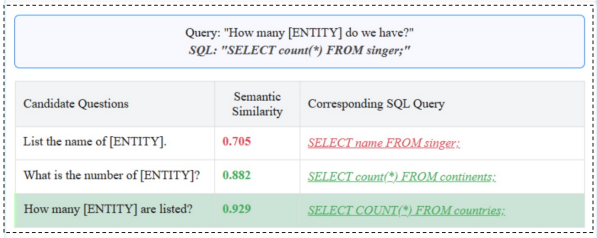
intent and the precision of SQL generation. To mitigate this issue, retrieval-augmented generation (RAG) and schema linking techniques can be employed to provide grounded context and enhance the model's ability to correctly interpret database structures and user queries.



Query: "How many singers do we have?"
SQL: "SELECT count(*) FROM singer"

Candidate Questions	Semantic Similarity	Corresponding SQL Query
List the name of singers.	0.681	SELECT name FROM singer
What is the number of continents?	0.517	SELECT count(*) FROM continents
How many countries are listed?.	0.608	SELECT count(*) FROM countries

(a) Relying solely on the embedding similarity of questions to select examples may overlook correct examples and instead retrieve incorrect ones.



Query: "How many [ENTITY] do we have?"
SQL: "SELECT count(*) FROM singer;"

Candidate Questions	Semantic Similarity	Corresponding SQL Query
List the name of [ENTITY].	0.705	SELECT name FROM singer;
What is the number of [ENTITY]?	0.882	SELECT count(*) FROM continents;
How many [ENTITY] are listed?	0.929	SELECT COUNT(*) FROM countries;

(b) Applying entity masking to the question can, to some extent, highlight its syntactic structure and help mitigate this issue.



Questions	Semantic Similarity	SQL Query
What is the highest [ENTITY] for [ENTITY] in the [ENTITY] in [ENTITY]?	0.893	SELECT 'Free Meal Count (K-12)' 'Enrollment (K-12)' FROM frpm WHERE 'County Name' = 'Alameda' ORDER BY (CAST('Free Meal Count (K-12)' AS REAL) 'Enrollment (K-12)') DESC LIMIT 1
What is the [ENTITY] of the [ENTITY] that has the highest number of [ENTITY] with an [ENTITY] of over [ENTITY]?		SELECT T2.Phone FROM satscores AS T1 INNER JOIN schools AS T2 ON T1.cds = T2.CDSCode ORDER BY T1.NumGE1500 DESC LIMIT 1

(c) Entity masking in complex queries can cause significant information loss, leading to high similarity between questions with different SQL structures

Fig. 1. Limitations of RAG in the Text-to-SQL

RAG provides LLM with a factual basis by retrieving relevant exemplars. However, existing RAG strategies, relying primarily on textual similarity, neglect critical schema and SQL syntactic structures. This text-centric approach retrieves structurally irrelevant examples that mislead the model rather than providing useful guidance. Questions can be textually similar yet correspond to vastly different SQL syntactic structures (Figure 1(a)). To address this limitation, the technique of entity masking can reduce dependency on specific schema names(Figure 1(b)), but it can also be misleading by erasing key information. As illustrated in Figure 1(c), even though two masked questions may appear textually similar, their corresponding SQL structures can differ substantially—one involve complex aggregation and computation, while the other is a simple filtering query.

This highlights the insufficiency of text-based similarity. Moreover, although entity masking mitigates schema dependency, it also removes essential semantic information, reducing the embedding's

fidelity in reflecting SQL intent. Since neither text-based retrieval nor entity masking adequately preserves SQL structural alignment, this study asks: can a structure-aware representation be learned to retrieve syntactically coherent exemplars for generation?

However, retrieving structurally-sound exemplars is not sufficient on its own. The query generation process is fundamentally reliant on two pillars: a correct syntactic structure and a correct mapping of semantic intent to the schema. While a structure-aware retriever addresses the first, the second—accurately identifying which tables and columns to use—remains a distinct and critical bottleneck.

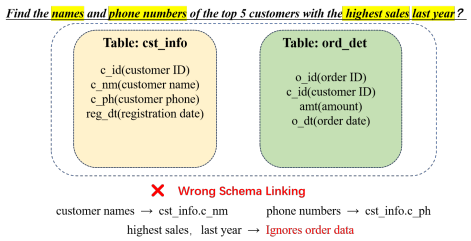


Fig. 2. Limitations of schema linking in the Text-to-SQL

Schema linking thus becomes a crucial component in addressing this bottleneck and mitigating hallucination in Text-to-SQL tasks. Existing methods typically formulate schema linking as a classification problem—using models such as BERT with a classification head [19]—or leverage the few-shot learning capabilities of LLMs [10]. However, these approaches often struggle to align natural language queries with abbreviated or non-standardized schema names. While they can successfully match simple fields like customer name, they frequently fail to capture more complex semantic constructs such as highest sales or last year, which require deeper reasoning and relational understanding across tables like orders (Figure 2).

This research, therefore, poses the second key question: Can the integration of Chain-of-Thought (CoT)[36] reasoning and Reinforcement Learning (RL)[24] enhance schema linking to achieve a more refined and semantically consistent alignment, ultimately improving the structural grounding for SQL generation?

To address the above two key challenges of Text-to-SQL, this paper proposes a Schema-aware Retrieval-Augmented Generation (SchemaRAG) framework. It has three core components. First, SchemaLinker is fine-tuned to map natural language to schema elements by leveraging knowledge distilled from high-quality CoT examples. Its reasoning capabilities are further enhanced through group relative policy optimization. Second, a Schema-Aware Retriever (SAR) is introduced to select the most relevant examples by utilizing the structure of the database schema, thereby improving the LLM's understanding and generation of SQL syntax. Finally, SchemaRAG designs a Pareto-Optimal SQL Generator (POSG) to choose the best SQL query from a pool of high-quality candidates, increasing overall robustness. Together, these components enable SchemaRAG to better align complex schemas with SQL syntax, improving query accuracy. To evaluate the proposed SchemaRAG, we adopt several mainstream LLMs as the base models and compare with four state-of-the-art Text-to-SQL methods. The experimental results on two benchmark datasets show that our SchemaRAG achieves significantly better performance than its peers.

The primary contributions of our work are as follows:

- **SchemaLinker:** A module that achieves high-precision schema linking through a rigorously optimized training procedure.

- **SAR:** A novel retriever that effectively utilizes database schema structures to learn more accurate, SQL-syntax-aware representations within question embeddings.
- **SchemaRAG framework:** A novel architecture presented with theoretical analyses and algorithm designs.
- **Comprehensive experiments:** Conducted on the benchmark datasets of Spider, BIRD, Spider 2.0 Snow, Beaver, and UniSQL, demonstrating consistent outperformance over competitive baselines across multiple evaluation metrics, highlighting superior generalization and robustness.

2 Related Works

2.1 Text-to-SQL based on Language Models

Before the advent of LLMs, text-to-SQL generation predominantly relied on fine-tuning encoder-decoder architectures. Early studies enhanced encoder representations by incorporating structural relations among query tokens, tables, and columns via Graph Neural Networks[4, 35], while others improved decoders by injecting SQL grammar to ensure syntactic correctness [9, 30, 38]. With the rise of Pre-trained Language Models such as T5 and BART [18, 29], fine-tuning on input-output pairs became the mainstream paradigm [8]. The emergence of competitive LLMs, including GPT [3], Llama [34], Qwen [2], and DeepSeek [6], has revolutionized NLP, particularly tasks requiring complex reasoning. Consequently, research has shifted toward leveraging LLMs for Text-to-SQL through various strategies: task decomposition and few-shot prompting [26], zero-shot prompt engineering [7], incremental pretraining [20], critic-based selection [12], and CoT reasoning integration [27, 41]. FinSQL [40] further proposes a model-agnostic approach combining prompt construction, parameter-efficient fine-tuning, and output calibration for SQL generation. However, these methods often struggle with complex schema alignment and SQL accuracy.

To overcome these limitations, we propose SchemaRAG. It integrates SchemaLinker, which maps natural language to schema elements using distilled CoT knowledge; SAR, which selects exemplars by encoding schema structures via cross-attention and contrastive learning; and POSG, which ensures robust SQL query selection.

2.2 Text-to-SQL with RAG

RAG has shown considerable promise in addressing the Text-to-SQL problem. However, retrieval strategies based on surface-level question similarity often fall short, as textually similar questions may correspond to substantially different SQL structures. Recent research therefore incorporates SQL structural information to enhance retrieval. For instance, ACT-SQL [41] constructs and selects appropriate CoT exemplars for LLMs via RAG, while ReFSQL [42] employs the Tree-Edit-Distance between SQL queries as a contrastive learning signal to refine question representations. Similarly, ASTRES [33] generates an approximate SQL query and re-ranks candidates based on Abstract Syntax Tree (AST) similarity. Despite these advancements, their reliance on surface-level or sequential modeling still constrains effectiveness.

To overcome these limitations, we propose SAR, a novel representation learning framework designed to produce schema-aware embeddings. By fusing question and schema information through cross-attention and optimizing via contrastive learning, SAR directly encodes SQL structural properties to enable more precise and semantically grounded retrieval.

2.3 Text-to-SQL with RL

LLMs offer a promising platform for Reinforcement Learning (RL), which optimizes objectives through interaction-based learning. Foundational methods such as DQN [24], PPO [31], and Actor-Critic models [14] have established the basis for integrating RL with LLMs. To alleviate the computational burden of PPO, more efficient variants like Group Relative Policy Optimization have been introduced, as demonstrated in DeepSeekMath [32]. In the Text-to-SQL domain, however, the application of RL remains limited by the challenge of sparse rewards, particularly for complex or deeply nested queries. Prior work, such as SQL-R1 [23], has sought to address this by introducing multiple reward mechanisms, yet models still struggle to learn effective policies from limited positive feedback.

To effectively address this challenge, our research breaks down the complex process of end-to-end SQL query generation into distinct, manageable components, with a specific focus on enhancing the schema linking sub-task. We apply RL techniques to this targeted sub-task to improve the accuracy and efficiency of aligning natural language queries with the appropriate database schema elements.

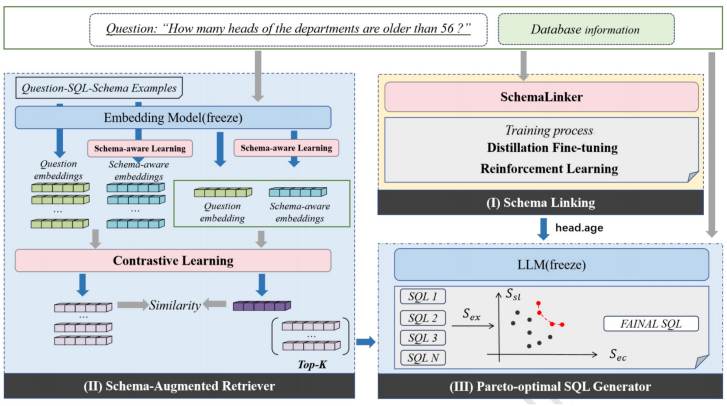


Fig. 3. The SchemaRAG architecture consists of three interconnected modules: the SAR for semantic retrieval and contextualized generation, the SchemaLinker for aligning query entities with schema elements, and the POSG for synthesizing the final robust query.

3 The Proposed SchemaRAG Framework

3.1 The Overall Structure

$$S = L(Q, D) \tag{1}$$

The Text-to-SQL task, as shown in Equation 1, translates a natural language question into an executable SQL query based on the database schema (e.g., table names, column names, data types) to accurately capture user intent and retrieve correct results. To reduce hallucination issues in this translation process, our proposed SchemaRAG framework, illustrated in Figure 3, integrates three complementary and synergistic components designed to enhance SQL query generation.

(1) **SchemaLinker:** The first component is the SchemaLinker, designed to facilitate SQL generation by aligning natural language with schema elements. We fine-tune Qwen-7B in three stages: we first distill CoT data from GPT-4o, then perform multi-task alignment, and finally optimize the model with reinforcement learning.

(2) **SAR:** The second component is a mechanism designed to enhance example selection, taking as input the database schema and the user question. An embedding model first processes these inputs, followed by a cross-attention mechanism to generate Schema-Aware embeddings. Contrastive learning is then applied to both the question embeddings and the Schema-Aware embeddings to improve their representation quality and increase discriminability. Finally, similarity scores are computed based on the enhanced representations to retrieve the top-K most relevant Text-SQL examples.
(3) **POSG:** The third component is a SQL generation and selection module. It first prompts the LLM to generate multiple candidate SQL queries, which are then subjected to multi-dimensional evaluation. Based on these evaluations, the Pareto optimality principle is applied to identify the final, most optimal SQL query.

3.2 SchemaLinker

To enhance the accuracy and mitigate hallucinations in Text-to-SQL tasks (Equation 1), we introduce SchemaLinker through four key stages: PromptSchema, CoT-aligned training, error-driven multi-task alignment, and RL with GRPO.

Table 1. Ambiguous Column Names and Clarifications

Original Name	Potential Ambiguity	Required Meaning
reg_dt	Regulation date? Region ID?	Customer registration date
c_nm	City name? Company name?	Customer's full name
amt	Amount of what? Quantity?	Order amount (in USD)
id	Which ID? Ambiguous.	The singer's unique ID (Primary Key)

3.2.1 Automatically PromptSchema. The examples in Table 1 illustrate how ambiguous column names, such as reg_dt or c_nm, can lead to multiple interpretations and hinder accurate schema understanding. These cases highlight the need for a systematic approach to clarifying column semantics, rather than relying on ad hoc manual annotations. To address this foundational challenge of how LLMs comprehend database schemas, we propose PromptSchema. Our approach integrates the BM25S algorithm [22], which **automatically** extracts representative data samples for each column (details in Appendix K), ensuring a fully automated and deterministic schema interpretation process. This data-grounded foundation enables the LLM to form a stable and semantically rich understanding of schema elements (details in Appendix H.1).

3.2.2 Ensuring CoT Aligns with Ground Truth. Building upon this structured schema representation, we developed SchemaLinker by fine-tuning the Qwen-7B model [37]. Qwen-7B was selected as the base model due to its strong Text-to-SQL performance [15] and optimal balance between capability and computational efficiency. Through targeted fine-tuning, SchemaLinker learns to resolve schema ambiguities like those shown in Table 1, effectively aligning natural language queries with the correct schema components. This enhanced schema linking ability further guides accurate SQL generation (Equation 1) and improves model robustness across large-scale, real-world databases.

The training process for our SchemaLinker, illustrated in Figure 4, begins with generating high-quality CoT rationales from our teacher model (GPT-4o), ensuring strict alignment with the ground truth. To achieve this, we employ a multi-faceted verification approach, as outlined in Equation 2, where the ground-truth SQL query is included in the prompt to guide the teacher model in referencing the correct SQL logic while generating the schema-linking rationale. Post-generation, we apply a rigorous filtering process: CoT outputs with final answers mismatching the ground-truth label are discarded, and a script parses the remaining rationales to extract all

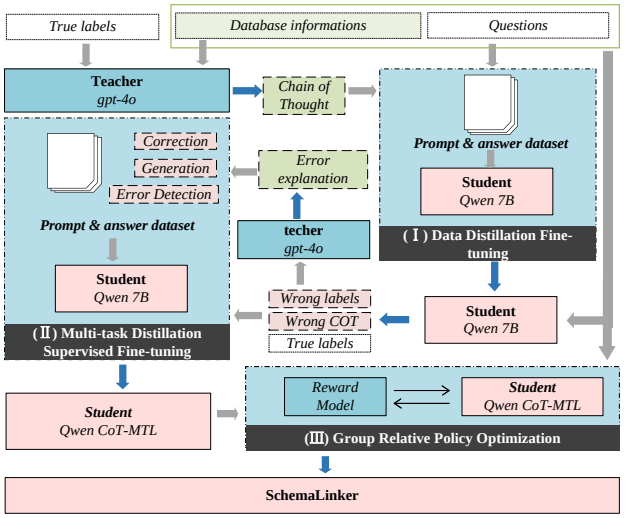


Fig. 4. **The training process of the schema linking model progresses through three key stages: initial CoT distillation (top right), followed by multi-task distillation (left), and culminating in GRPO-based reinforcement learning (bottom right), ultimately resulting in the SchemaLinker model.**

mentioned database entities (tables and columns), verifying their perfect consistency with those used in the corresponding SQL query. Only rationales passing this strict entity-level consistency check are retained for training, with the CoT generation process detailed in Appendix J.2.

$$\text{For each } (x_i, y_i) \in \{(x_i, y_i)\}_{i=1}^{N} \tag{2}$$

$$(CoT_i, \hat{y}_i) = M_{teacher}(x_i, y_i) \tag{3}$$

As shown in Equation 3, the CoT and the predicted label are generated by the teacher model.

$$\text{If } \hat{y}_i = y_i \text{ Then } \mathcal{D} \leftarrow \mathcal{D} \cup \{(x_i, CoT_i, y_i)\} \tag{4}$$

Following the process in Equation 4, we store the resulting CoT and its corresponding predicted label as a single data instance, after confirming their consistency with the ground-truth label. After this initial training, the fine-tuned Qwen-CoT model was evaluated for five epochs on the rest of the Spider training set. A candidate error set was compiled by taking the union of all samples from each epoch where the predicted labels of the model mismatched the ground-truth labels. However, we observed that in some instances of label mismatch, the model could still generate the correct final SQL query. This necessitated a subsequent manual curation of the candidate set to isolate samples that led to genuinely incorrect SQL outputs, thus forming our refined error dataset. This dataset was then annotated with reasoning paths generated by GPT-4o, which were subsequently corrected and used for a second stage of fine-tuning.

3.2.3 Error-Driven Multi-Task Alignment. Inspired by an error-driven learning paradigm, we propose a multi-task alignment framework designed to enhance the primary task performance by explicitly training it to recognize and rectify its own deficiencies. As shown in Figure 5, this framework comprises three objectives:

(1) **Error Detection:** Identifying and explaining errors within the reasoning process and schema linking, with detailed error types provided in Appendix H.2.
(2) **Corrective Analysis:** Generating the correct reasoning path and final answer based on the error analysis.
(3) **Answer Generation:** Directly producing the correct reasoning path and answer from the natural language question and database schema.

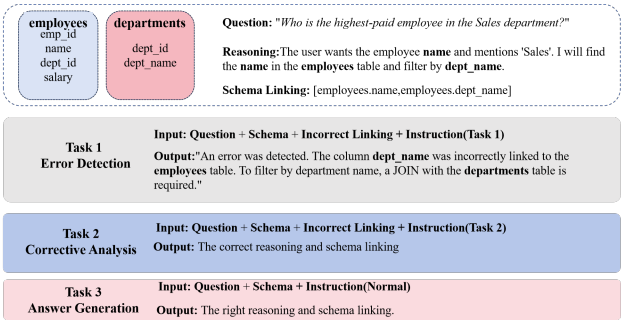


Fig. 5. An example of multi-task alignment.

The goal of this approach is to enhance the primary task proficiency of the model by explicitly training it to recognize and rectify the deficiencies of the model itself, c.f., Appendix H.2, which details the types of errors corrected and the expected output format.

$$\hat{x}_n = [\, I_{i(n)} \parallel x_n \,], \tag{5}$$

First, as shown in Equation 5, the input sequence, $\hat{x}_n$, is formed by concatenating a specific task instruction, $I_{i(n)}$, with the original input x_n.

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P_\theta\big(y_t \mid \hat{x}_n, y_{<t}\big). \tag{6}$$

Then, given the predicted probability distribution of the model, the cross-entropy loss for a single task instance n is expressed as:

$$p_n = \frac{m_{i(n)}}{\sum_{j \in \mathcal{T}} m_j} \cdot \frac{1}{N_{i(n)}}, \tag{7}$$

To guide the model's learning toward more important tasks, we adopt a weighted sampling strategy. As shown in Equation 7, the sampling probability p_n of each instance depends on the task priority weight $m_{i(n)}$ and the number of instances $N_{i(n)}$ in its task. The weights $m_{i(n)}$ determine task importance, normalized over all tasks $\mathcal{T}$, while division by $N_{i(n)}$ evenly distributes this weight across instances. In practice, we set task priorities to a 3:3:10 ratio to balance training across tasks of different difficulty levels.

$$E[\text{samples}_i] \propto N_i \cdot p_n, \tag{8}$$

Consequently, as shown in Equation 8, the expected number of samples for each task i, denoted $E[\text{samples}_i]$, is proportional to the number of instances in that task N_i and the instance sampling probability p_n.

3.2.4 RL for Schema Linking. Finally, we set a uniform number of samples for each task. Consequently, the task sampling frequency is controlled solely by our custom-defined weights, enabling the model to focus its learning on high-priority tasks.

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{\pi_{\text{old}}(o|s)} \Bigg[\frac{1}{G} \sum_{o_i, o_j \in O} \min \left(r_i^{\text{ratio}} A_j,\ \text{clip}(r_i^{\text{ratio}}, 1-\epsilon, 1+\epsilon) A_j \right) - \beta D_{\text{KL}}(\pi_{\text{old}} \| \pi_\theta) \Bigg] \tag{9}$$

Considering that the schema linking task is analogous to mathematical reasoning—where a unique correct answer often exists—we employ the Group Relative Policy Optimization (GRPO) method for the final reinforcement learning fine-tuning of the large model. As shown in Equation 9, for each natural language question and its corresponding database schema, the policy model generates a candidate set of G schema combinations, $\{o_1, o_2, \ldots, o_G\}$, based on the old policy. These candidates are then meticulously evaluated against our defined reward rules to assign specific reward scores. By focusing on the relative performance of the schema links within the group, we guide the policy update according to our established objective:

$$R_k = \begin{cases} R, & \text{if format is correct} \\ 0, & \text{if format is incorrect} \end{cases} \tag{10}$$

To promote structured output, we guide the model to follow a predefined format for its responses. The intermediate reasoning steps, derived from prior training, are also formatted to ensure transparency and prevent internalization of computational steps. For training stability, as delineated in Equation 10, outputs that do not adhere to the specified format are assigned a zero reward. Consequently, incorrectly formatted outputs preclude the model from receiving any subsequent reward components. Detailed format specifications are provided in Appendix H.2.

$$R = r_c \cdot |P_s \cap T_s| + r_w \cdot |P_s - T_s| + r_m \cdot |T_s - P_s| + w_f \cdot F_1 \tag{11}$$

As shown in Equation 11, the reward function R evaluates schema linking performance as a weighted sum of four components. P_s and T_s denote the predicted and true schema items, respectively. The three set-based terms represent True Positives (TP), False Positives (FP), and False Negatives (FN), while F1 score is the harmonic mean of precision and recall. In Text-to-SQL, FN (*missed schema items*) are more detrimental than FP, as missing a required table or column prevents correct SQL generation, while redundant items can often be ignored by the decoder. Hence, the reward emphasizes recall through stronger FN penalties. The weight configuration is as follows:

- $r_m = -3$: strong penalty for FN;
- $r_c = +2$: reward for TP;
- $r_w = -0.5$: mild penalty for FP;
- $w_f = +0.5$: bonus for the overall **F1 score**.

This reward design encourages comprehensive schema coverage while maintaining reasonable precision. The trained SchemaLinker, optimized under this scheme, demonstrates strong performance in accurately identifying relevant tables and columns for downstream applications[25]. For detailed evaluation results—including training configuration, token usage, ablation studies, and comparisons across different training stages—please refer to Appendix B.

3.3 Schema-Augmented Retriever

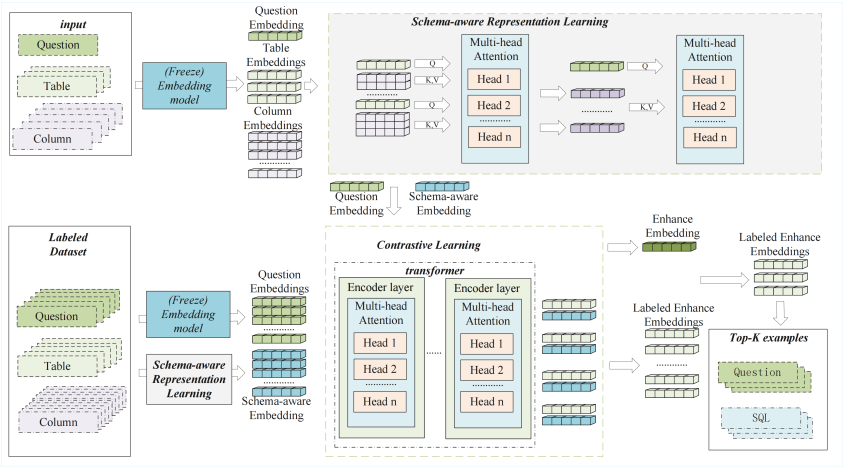


Fig. 6. The RAG process involves generating Schema-Aware embeddings via cross-attention and producing enhanced representations with a Transformer, which are then used to retrieve top-k question-SQL examples for few-shot learning.

As shown in Figure 6, to enhance the LLM's ability to generate syntactically correct SQL statements, we adopt the RAG methodology and provide it with high-quality, Schema-Aware few-shot examples. These examples are generated by leveraging GPT-4o in conjunction with our original Question-SQL-Schema pairs, ensuring that the retrieved instances are both semantically relevant to the input question and syntactically consistent with the target SQL structure.

3.3.1 *Schema-Aware Representation Learning .*

$$c_j^i = \text{Encoder}(\text{col_name}_j^i \parallel \text{col_description}_j^i) \tag{12}$$

$$t_i = \text{Encoder}(\text{table_name} \parallel \text{table_description}_j^i) \tag{13}$$

$$E_{q_n} = \text{Encoder}(\text{question}_n \parallel \text{evidence}_i^{(\text{opt})}) \tag{14}$$

$$E_{s_n} = \text{Encoder}(\text{SQL}_n) \tag{15}$$

First, textual information from the database schema, the user's question, and the corresponding SQL query are transformed into numerical embeddings in a shared embedding space. Each database column and table, along with their respective names and descriptions, is encoded into an embedding (Equation 12–13). Similarly, the user's question, which can be supplemented with additional evidence, is also converted into an embedding (Equation 14). Finally, the SQL query itself is processed by the same encoder to generate its embedding (Equation 15).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{16}$$

The Schema-Aware embedding learning model is fundamentally based on the attention mechanism. It computes similarity scores between a query and a set of keys to determine the degree of "attention" that should be assigned to the corresponding values. The computation process is formally defined in Equation 16.

$$T = \{t_1, t_2, \ldots, t_n\} \tag{17}$$

$$C = \{c_1^1, \ldots, c_{j_1}^1, c_1^2, \ldots, c_{j_2}^2, \ldots, c_1^n, \ldots, c_{j_n}^n\} \tag{18}$$

First, as shown in Equations 17 and 18, the database consists of a finite set of relations (tables), each of which is characterized by a specific set of attributes (i.e., column names).

$$T_i^C = \text{Attention}(T_i, C_:^i, C_:^i) \tag{19}$$

To enrich each table embedding with its internal structure rather than treating it as an isolated token, we introduce an attention mechanism. As shown in Equation 19, each table T_i serves as a query attending to its own columns $C_:^i$, which act as both keys and values, yielding a column-aware table embedding T_i^C.

$$\hat{S}_i = \text{Attention}(E_{q_i}, T_:^C, T_:^C) \tag{20}$$

As shown in Equation 20, after obtaining table embeddings enriched with column-level information, the next step is to associate them with the specific user question. We perform a second attention operation, where the embedding of the user question serves as the query and attends to all column-aware table embeddings, which act as the keys and values.

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left\| E_{s_i} - \hat{S}_i \right\|_2^2 \tag{21}$$

The final embedding is a highly condensed representation that captures the structural information of the database. The goal of training is to ensure that the predicted embedding closely matches the embedding of the ground-truth SQL query in the vector space (Equation 21).

3.3.2 Contrastive Learning. Upon completion of the training of the Schema-Aware embedding model, we can generate Schema-Aware embeddings based on the input question and schema information. However, these initial schema-aware embeddings still have a key limitation: entity-specific details introduce variance that prevents semantically similar queries from clustering effectively in the embedding space (Figure 12(a)). Specifically, entity-level information refers to the concrete nouns and values within a query, such as "Sales" in "Show employees in the Sales department". Two queries might share the same underlying intent (e.g., listing employees by department) but differ only by the entity ("Sales" vs. "Marketing"), causing their embeddings to be undesirably distant. Consequently, relying solely on these Schema-Aware embeddings for similarity comparison yields suboptimal results.

To address this limitation, we introduce an embedding enhancement model based on contrastive learning. This approach learns a more robust representation by constructing positive and negative sample pairs. Specifically, a positive pair consists of questions from our RAG_train dataset that share similar SQL syntax structures. For negative pairs, we employ an in-batch sampling strategy, where for any given anchor sample within a batch, all other samples in that batch are treated as its negative samples.

$$X^{(0)} \in \mathbb{R}^{2 \times b \times d} \tag{22}$$

To begin, as shown in Equation 22, we stack the question embedding and the Schema-Aware embedding to form an input sequence of length two.

$$M = \begin{pmatrix} 0 & 0 \\ -\infty & 0 \end{pmatrix} \tag{23}$$

Next, as shown in Equation 23, to regulate the flow of information between the question and schema embeddings, we apply a structured causal mask M, within the self-attention mechanism of the Transformer.

This causal mask enforces a unidirectional information flow: the question embedding can attend to the schema embedding, but not vice-versa. This design allows the schema to inform the question's representation without reciprocal distortion.

$$X^{(l)} = \text{TransformerLayer}(X^{(l-1)}, M) \quad \text{for } l = 1, \ldots, L \tag{24}$$

As shown in Equation 24, the input sequence is then processed through a series of L masked Transformer encoder layers to produce a final output sequence $X^{(L)}$.

$$E_{\text{final}} = \text{LayerNorm}(X^{(L)}[0, :, :]) \in \mathbb{R}^{b \times d} \tag{25}$$

As shown in Equation 25, from this final output sequence, we extract the first vector, which corresponds to the newly enhanced question representation. A final layer normalization is applied to produce the enhanced embedding E_{final}.

To train the model and ensure the separability of augmented embeddings from diverse initial pairs, the model incorporates a contrastive learning objective. Furthermore, to promote proximity for augmented embeddings derived from known similar initial pairs, the model also explicitly employs a similarity objective.

$$E_{\text{main}} = f(E_Q\text{main}, \hat{S}_{\text{main}}) \in \mathbb{R}^{b \times d} \tag{26}$$

$$E_{\text{similar}} = f(E_Q\text{sim}_k, \hat{S}_{\text{sim}_k}) \in \mathbb{R}^{b \times d} \quad \text{for } k = 1, \ldots, N_{\text{sim}} \tag{27}$$

Specifically, as shown in Equations 26 and 27, both the anchor sample embedding and the similar sample embedding are fed into the model to generate their respective enhanced embedding representations.

$$L_{\text{contrastive}} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp\left(\text{sim}(z_i, z_i^+)/\tau\right)}{\sum_{j=1}^{N} \exp\left(\text{sim}(z_i, z_j)/\tau\right)}, \tag{28}$$

To achieve this, as shown in Equation 28, we employ a contrastive loss function that treats other samples within the same batch as negative examples and minimizes their similarity with the anchor sample to enhance the discriminative capability of the model.

$$\mathcal{L}_{\text{similarity}} = \frac{1}{N_{\text{sim}} \cdot N} \sum_{k=1}^{N_{\text{sim}}} \sum_{i=1}^{N} \left(1 - \cos(E_{\text{main}}^{(k,i)}, E_{\text{similar}}^{(k,i)})\right), \tag{29}$$

In addition, as shown in Equation 29, a similarity loss term is introduced to minimize the distance between the enhanced embedding of the anchor sample and that of its corresponding similar sample, thereby promoting semantic consistency.

$$L_{\text{total}} = L_{\text{contrastive}} + w_{\text{sim}} \cdot L_{\text{similarity}} \tag{30}$$

Therefore, as shown in Equation 30, the final total loss is computed as a weighted sum of the contrastive loss and the similarity loss, with a fixed similarity loss weight $w_{\text{sim}} = 0.5$. This combined loss function is designed to intentionally leverage two complementary objectives, each operating on differently structured data. The contrastive loss enhances discriminability by treating samples within a general batch as negative examples for one another. Concurrently, the similarity loss

promotes consistency by operating on a curated dataset of "known similar initial pairs," pulling their embeddings closer. This dual-objective approach, utilizing information from both general and paired data structures, effectively balances the trade-off between discriminability and semantic alignment to learn a more robust representation space.

In conclusion, our proposed method integrates a two-stage process to optimize the retrieval of few-shot examples for Text-to-SQL generation. The first stage creates an initial Schema-Aware representation by encoding the database schema's structure relative to the user's question, training it to align with the ground-truth SQL query's embedding. The second stage employs a contrastive learning framework. It takes a combined input of the original question embedding and the Schema-Aware representation, processing them through a masked Transformer to produce a final, enhanced embedding. This contrastive training makes the final representation highly discriminative of the shared SQL syntactic structure between queries, while remaining invariant to superficial, entity-level variations. By leveraging these enhanced embeddings, we accurately retrieve the top-k most similar Question-SQL pairs, providing the LLM with high-quality, contextually-aware examples that guide it to generate SQL queries that are both syntactically correct and precisely aligned with user intent.

3.4 Pareto-optimal SQL Generator

```
Question: How many singers do we have?
database .....
schema used: [singer]

In-Context Learning Examples

Question: How many employees are currently working?
SQL: SELECT count(*) FROM Employees

Question: What is the total number of employees?
SQL: SELECT count(*) FROM EMPLOYEES

Question: What is the total number of students enrolled?
SQL: SELECT count(*) FROM Students
```

Fig. 7. Example of the final Text-to-SQL prompt.

After processing the schema linking and example augmentation tasks, we input the schema links, PromptSchema, and the selected examples into the LLM to generate a set of candidate SQL queries. These components are combined into a single prompt, the specific format of which is shown in Figure 7. To further enhance the precision of SQL generation and fully exploit the vast decoding space of LLMs, we perform a multi-dimensional evaluation of the candidate queries. We then apply the Pareto optimality principle to select the most appropriate SQL query based on its executability (S_{ex}), schema linking conformity (S_{sl}), and example consistency (S_{ec}).

$$S_{ex}(S) = \begin{cases} 1 & \text{if } S \text{ is executable without errors} \\ 0 & \text{if } S \text{ is not executable} \end{cases} \tag{31}$$

$$Examples = \{e_1, e_2, ..., e_n\} \tag{32}$$

In the multi-dimensional evaluation, we first apply a strict filtering step to all generated SQL queries. This step is based on executability (S_{ex}), as defined in Equation 31. Any query that fails to

execute successfully on the database (i.e., $S_{ex}(S) = 0$) will be discarded directly. Then, the remaining valid candidate queries (compared against the few-shot set in Equation 32) are assessed for schema conformity and syntactic consistency. For the set of SQL queries that pass the executability check, we subsequently evaluate their reliability using the Pareto optimal principle across two dimensions.

$$S_{sl}(S, S_k) = \frac{|SchemaUsed(S) \cap S_k|}{|SchemaUsed(S) \cup S_k|} \tag{33}$$

For the set of SQL queries that pass the executability check, we subsequently evaluate their reliability using the Pareto optimal principle across two dimensions. The first dimension is schema linking conformity (S_{sl}). This score is calculated based on Equation 33 and measures the Jaccard similarity between the schema used in the query and the linked schema.

$$S_{ec} = \frac{1}{n} \sum_{i=1}^{n} \left(1 - \frac{d(\text{AST}(S), \text{AST}(e_i))}{\max(|\text{AST}(S)|, |\text{AST}(e_i)|)}\right) \tag{34}$$

The second dimension is example consistency (S_{ec}). Its calculation is given in Equation 34, which is based on the average tree edit distance of the abstract syntax tree (AST) between the generated query and the example sample set.

$$\begin{aligned} P = \{S_i \in E \mid \neg\exists S_j \in E \text{ s.t. } (S_{sl}(S_j) \geq S_{sl}(S_i) \wedge S_{ec}(S_j) \\ \geq S_{ec}(S_i)) \wedge (S_{sl}(S_j) > S_{sl}(S_i) \vee S_{ec}(S_j) > S_{ec}(S_i))\} \end{aligned} \tag{35}$$

We then identify the Pareto-optimal set P, which contains all non-dominated queries.A query S_i is considered non-dominated if no other candidate query S_j exists that performs equally or better on all evaluation metrics and strictly better on at least one, as formally defined in Equation 35. This set contains a group of high-quality candidate queries that achieve different "trade-offs" between schema linking conformity and example consistency.

In summary, this approach enables us to identify a set of candidate queries that, while not superior or inferior, each offer distinct advantages. This ensures that the final selected SQL query is not only executable but also optimal or near-optimal in its core quality dimensions.

3.5 Algorithm Design

According to the above analyses, we designed the *Algorithm SchemaRAG*. Its pseudocode is presented and analyzed in Algorithmic. 1.

4 Experiments

In the subsequent experiments, we aim at answering the following research questions (RQs):

- RQ.1. Does SchemaRAG achieve the best performance among the baseline models considered in this study?
- RQ.2 What types of errors occur in SchemaRAG?
- RQ.3. How does Self-Consistency impact schema linking in SchemaLinker, and what are the key factors affecting the maintainability and efficiency of SchemaRAG?
- RQ.4. What factors influence the performance of SAR, and to what extent is the model itself effective?
- RQ.5. Are all the modules in our designed SchemaRAG farmework necessary for the model to function effectively?

Algorithm 1 The overall algorithm workflow of SchemaRAG

Inputs: $Q, DB, Examples$
Output: S
Sample $\leftarrow$ BM25S(Q, DB)
$D \leftarrow$ PromptSchema(Sample, DB)
$S_k \leftarrow$ SchemaLinker(Q, D)
procedure ProcessDataSource(embedding_args...)
    $(T_{out}, C_{out}, Q_{out}) \leftarrow$ Embedding(embedding_args...)
    $T'_{out} \leftarrow$ SchemaAware$(T_{out}, C_{out}, Q_{out})$
    $E \leftarrow$ BuildEnhance(T'_{out}, Q_{out})
    return E
end procedure
$E \leftarrow$ ProcessDataSource(DB, Q)
$E_{example} \leftarrow$ ProcessDataSource(Examples)
$\text{Top}_k \leftarrow$ Similar$(E, E_{example})$
$\{S_1, S^1_{gen}, ..., S^k_{gen}\} \leftarrow$ Generation$(Q, \text{Top}_k, S_k, D)$
Score$(S^i_{gen}) \leftarrow (S^i_k, S_i, S_c)$
$S_{final} \leftarrow$ SelectFromDominatingSet$(\{S^1_{gen}, ..., S^k_{gen}\})$

4.1 General Settings

4.2 Experiment Setup

4.2.1 Datasets. The core experiments of this study were conducted on five authoritative benchmark datasets: Spider[39], BIRD[21], Spider 2.0 snow[17], Beaver[5], and UniSQL, selected to ensure robust generalization across diverse query types and database structures. While datasets like Spider emphasize complex cross-domain semantic parsing, BIRD focuses on large-scale database grounded evaluation in big data environments, Spider 2.0 snow highlights real-world enterprise workflows in cloud systems, and Beaver concentrates on private enterprise data warehouses with intricate schemas and business-oriented queries, UniSQL is specifically utilized as a comprehensive generalization benchmark. It is constructed via stratified sampling across four datasets, and is carefully balanced to ensure diversity in query and database complexity, providing a robust evaluation of generalization across various SQL query types and database structures. For the RAG module, we extended the official validation/test sets using GPT-4o to generate structurally similar "Question–SQL–Schema" triples, which were rigorously verified for structural and semantic consistency(details in Appendix J.3). A detailed description of these datasets is provided in Appendix C.

4.2.2 Evaluation Metrics. Evaluation metrics center on **Execution Accuracy (EX)**, which measures semantic correctness by comparing the execution results of predicted and ground-truth SQL across all datasets. For the **Spider** dataset, we report **Exact Match (EM)**, requiring strict string-level consistency to assess syntactic accuracy. For **BIRD**, **Beaver**, **UniSQL**, and **Spider 2.0 snow**, we rely primarily on *EX*, as their large-scale databases minimize false positives and ensure reliable evaluation. The **Valid Efficiency Score (VES)** is also used for **BIRD** to assess efficiency and validity of generated queries. Besides, **Recall** and **Matthews Correlation Coefficient (MCC)** are adopted. Details are provided in Appendix B.2.

4.2.3 Baselines. This study adopts a range of foundation models, including open-source models such as CodeS[20] (*PACMMOD 2024*), XiYanSQL[10] (*Alibaba 2025*), Qwen[37] (*Alibaba 2025*), and Llama [13] (*Meta AI 2024*), as well as powerful proprietary models like GPT-4o-mini and GPT-4o[1] (*OpenAI 2024*), DeepSeek-V3[6] (*DeepSeek 2024*) and GLM-4[11] (*Zhipu AI 2024*). These models are compared against competitive Text-to-SQL methods, including TA-SQL[28] (*ACL 2024*), Din-SQL[26]

(*NeurIPS 2023*), ACT-SQL[41] (*EMNLP 2023*), and C3-SQL[7] (*arXiv 2023*). Detailed information about each model can be found in Appendix E.

4.2.4 Implementation Details. Our experiments were conducted on a server equipped with two NVIDIA V100 32GB GPUs, 1024GB of RAM, and an Intel(R) Xeon(R) Gold 5218 CPU @ 2.30GHz, running Ubuntu 22.04 LTS with PyTorch 2.7.0. We utilized SQLite for all database management. Our embedding model is bge-large-en-v1.5. In our RAG framework, the top-k was set to 3 for retrieval. For the generation phase, we employed beam search to generate five SQL candidates when using open-source LLMs. For closed-source LLMs, we instead adopted a multi-sampling strategy, generating multiple SQL candidates through repeated stochastic decoding. To ensure that the final SQL output achieves an optimal balance across multiple quality dimensions, we adopt the principle of Pareto Optimality, deterministically selecting the final query from a set of high-quality, non-dominated candidate solutions.

4.3 Performance Comparison (RQ. 1)

Table 2. Model performance on benchmark datasets. The last two columns under the ⋆ model present results for SAR trained with BIRD-based (left) and Spider-based (right) contrastive learning data. • indicates a result higher than the corresponding value of SchemaRAG.

Model	LLM	Spider		BIRD		Spider 2.0	Beaver	UniSQL	
		EX	EM	EX	VES	EX	EX	EX	
TA-SQL	GPT-4o	85.00	68.70•	52.43	23.21	1.65	0	35.16	
	Deepseek-v3	84.60	75.80•	56.24	25.66	1.46	0	37.01	
	Qwen-7B	76.40	19.70	33.11	12.92	0	0	24.32	
	XiYanSQL-14B	76.60	33.60	37.74	17.42	1.65	2.40	26.95	
	Llama-8B	72.00	12.60	32.06	11.89	0	0	23.34	
DIN-SQL	GPT-4o	74.20	60.10	46.86	20.11	1.46	0	31.25	
	Deepseek-v3	80.60	43.70	47.11	18.81	1.46	0	32.03	
	Qwen-7B	73.80	27.30	27.50	9.67	0	0	21.29	
	XiYanSQL-14B	85.10	77.00	44.18	22.58	0.91	1.90	30.96	
	Llama-8B	73.50	27.90	26.98	10.67	0	0	20.90	
ACT-SQL	GPT-4o	73.40	45.10	47.51	22.15	1.46	0	31.45	
	Deepseek-v3	75.50	25.20	45.98	18.80	1.65	0	30.96	
	Qwen-7B	76.60	23.50	26.27	9.43	0	0	20.90	
	XiYanSQL-14B	75.80	23.70	43.16	20.35	1.46	1.44	29.49	
	Llama-8B	76.50	22.80	26.98	9.12	0	0	21.29	
C3-SQL	GPT-4o	81.90	46.90	44.46	18.86	1.46	0	30.76	
	Deepseek-v3	76.50	23.50	44.85	18.50	1.46	0	30.47	
	Qwen-7B	72.30	44.70	28.51	9.67	0	0	21.58	
	XiYanSQL-14B	78.30	48.80	42.34	22.10	1.83	1.44	29.49	
	Llama-8B	71.80	44.40	27.08	9.66	0.55	0	20.90	
SchemaRAG ⋆	GPT-4o	**85.60**	66.80	**68.90**	**31.98**	**15.90**	**7.18**	**48.05**	38.48
	Deepseek-v3	**89.60**	75.90	**68.38**	**31.05**	**10.79**	**6.67**	**46.88**	36.91
	Qwen-7B	**80.40**	**53.30**	**54.50**	**25.10**	**7.50**	**2.87**	**37.60**	28.13
	XiYanSQL-14B	**93.30**	**88.00**	**65.25**	**34.54**	**19.20**	**10.05**	**48.34**	38.28
	Llama-8B	**80.80**	**64.50**	**60.37**	**29.92**	**7.31**	**2.39**	**40.43**	30.76

Table 3. Loss/Win counts, Wilcoxon signed-rank test, and the Friedman test among models

Model	loss/win	p-value	F-rank
TA-SQL	152/8	1.34×10^{-5}	2.85
DIN-SQL	160/0	1.91×10^{-6}	3.45
ACT-SQL	160/0	1.91×10^{-6}	3.85
C3-SQL	159/1	1.91×10^{-6}	3.98
SchemaRAG	**9/631**	–	**1.05**

Table 2 presents the performance of the evaluated Text-to-SQL models on Spider, BIRD, Spider 2.0, Beaver, and UniSQL, with additional results in Appendix F. To ensure a comprehensive and statistically robust comparison, we conducted three analyses between SchemaRAG and baseline models, summarized in Table 3: (1) a Loss/Win analysis, counting cases where SchemaRAG outperforms baselines across EX, EM, and VES metrics; (2) the Wilcoxon signed-rank test to assess pairwise statistical significance; and (3) the Friedman test for overall model ranking, where a lower F-rank indicates better performance.

SchemaRAG achieves substantial gains over all baseline models (Table 2), with statistical validation confirming the significance of these improvements (Table 3). It attains a dominant win/loss ratio of 631/9, with all pairwise comparisons showing p-values below 0.05 under the Wilcoxon signed-rank test. On the UniSQL dataset, SchemaRAG (GPT-4o), trained with a SchemaLinker based solely on BIRD-based contrastive learning data, achieves an EX accuracy of 48.05%, surpassing the best baseline TA-SQL (35.16%). These results highlight SchemaRAG's strong robustness and generalization in complex, cross-domain Text-to-SQL tasks.

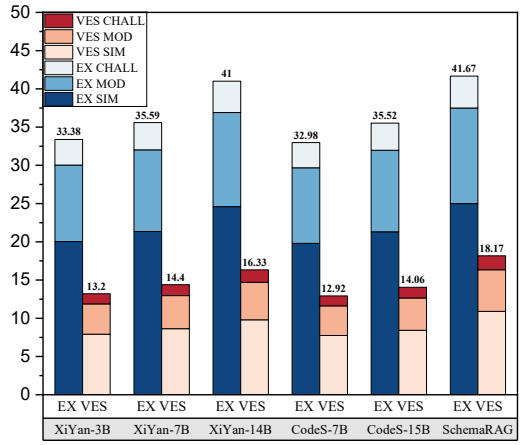


Fig. 8. A comparison with fine-tuned large models on the BIRD dataset without evidence.

Figure 8 shows the performance of various models, including the XiYanSQL series, CodeS series, and SchemaRAG(GLM-4-Plus), on the BIRD dataset without evidence, evaluated using both EX and VES metrics. We compare model performance across three difficulty levels—Simple (SIM), Moderate (MOD), and Challenging (CHALL)—under both metrics. SchemaRAG consistently achieves the highest scores across all categories: for EX, 50.38 (SIM), 29.89 (MOD), 23.61 (CHALL), and 41.67

(TOTAL); for VES, 22.33 (SIM), 11.82 (MOD), 11.86 (CHALL), and 18.17 (TOTAL). XiYanSQL-14B ranks second under the EX metric.

In summary, SchemaRAG(GLM-4-Plus) demonstrates superior performance overall and across all difficulty levels, significantly outperforming the fine-tuned models.

4.4 Error Analysis (RQ. 2)

To better understand its behavior and identify remaining challenges, we now present a detailed error analysis based on its implementation on GPT-4o. This analysis examines a random sample of 300 incorrect predictions generated by the model on the UniSQL dataset.

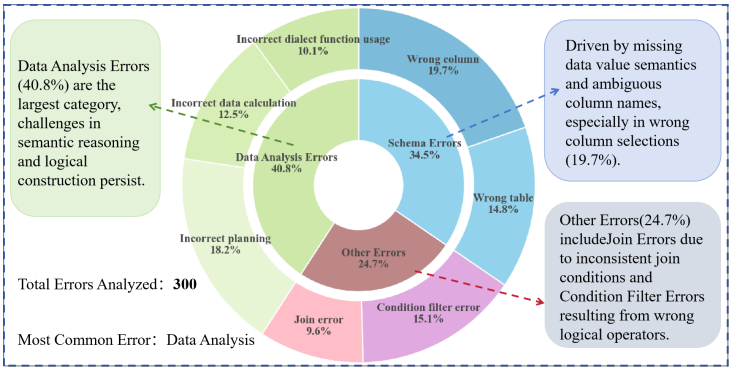


Fig. 9. Error statistics. Details of each error category are provided in Appendix I.

As illustrated in Figure 9, although our SchemaLinker mitigates schema-level issues and the SAR for SQL-aware question embeddings addresses higher-level semantic and logical reasoning challenges, the persistence of errors across schema linking, semantic understanding, and logical query construction still highlights the limitations of our approach in comprehending large, ambiguous databases.

4.5 Self-Consistency Effects on SchemaLinker and SchemaRAG Efficiency (RQ. 3)

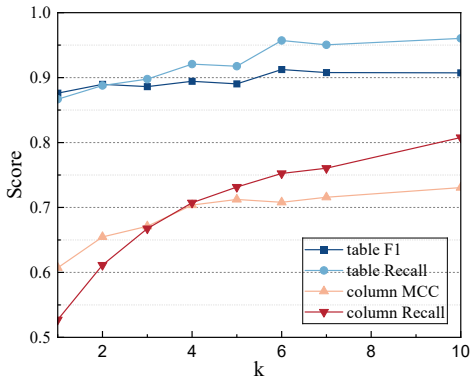


Fig. 10. Performance metrics (Table F1, Table Recall, Column MCC, and Column Recall) as a function of the number of self-consistency runs (k).

4.5.1 Impact of Self-Consistency on Schema Linking in SchemaLinker. As shown in Figure 10, to investigate the effect of self-consistency on the SchemaLinker, we conducted experiments on a challenging subset of 300 samples extracted from the BIRD dataset. To construct this challenging subset, we first ran our SchemaLinker model without the self-consistency strategy on the entire BIRD dataset. We then selected the 300 samples for which the model made errors in predicting key schema items (e.g., necessary table joins or columns). For each question, the SchemaLinker model was run k times. The final score was calculated based on the union of all results generated across these k iterations.

Our observations are as follows:

- As the value of k (i.e., the number of self-consistency trials) increases, all four evaluation metrics exhibit a clear upward trend. This indicates that leveraging a self-consistency strategy, by aggregating the union of multiple generated outputs, effectively enhances the performance of the SchemaLinker.
- The performance improvement is particularly significant as k increases from 1 to approximately 6 or 7. When k is further increased (e.g., beyond 7), the growth rate of the metric scores markedly decreases, and the curve begins to plateau. This suggests that in practical applications, excessively increasing the value of k may yield limited marginal gains, necessitating a careful balance between efficacy and computational cost.
- The improvement is most significant for column-related metrics because column selection is an inherently more complex task with a larger search space and lower initial accuracy. The strategy of taking the union of multiple outputs is highly effective at patching the omissions of individual runs, providing substantial recall gains for this more challenging sub-task.

Therefore, the experimental results confirm the effectiveness of the self-consistency method for improving the performance of the SchemaLinker model in extraction tasks, especially when processing high-difficulty samples.

Table 4. Maintainability and Efficiency analysis of different components on the UniSQL benchmark. **Green cells** indicate components with significantly increased context size or latency.

Configuration	**Context Size (tok/req)**	**Avg. Latency (s/req)**	**P95 Latency (s/req)**
	$L + \sigma$	$T + \sigma$	$T + \sigma$
GPT-4o	40 (-)	3.5 (-)	5.08 (-)
+ SchemaLinker			
+ *PromptSchema*	1710 (↑ 1670)	4.5 (↑ 1.0)	10.58 (↑ 5.50)
+ *fine-tuned*	3428 (↑ 1718)	11.4 (↑ 6.9)	25.65 (↑ 15.07)
+ *Self-consistency(5)*	10262 (↑ 6834)	36.7 (↑ 25.3)	128.45 (↑ 102.80)
+ SAR(3)	10289 (↑ 27)	37.1 (↑ 0.4)	128.58 (↑ 0.13)
+ POSG(5)(Final Model)	17345 (↑ 7056)	37.6 (↑ 0.5)	131.60 (↑ 3.02)

4.5.2 Maintainability and Efficiency Analysis of SchemaRAG. Our efficiency analysis on the UniSQL benchmark, based on incremental additions to GPT-4o, shows that the complete SchemaRAG framework effectively balances performance and maintainability, as summarized in Table 4. Early modules, such as PromptSchema and fine-tuned SchemaLinker, contribute moderate latency but provide essential functionality, while later components like SAR and POSG extend capabilities with minimal impact on processing time. This modular design allows each component to be tuned or replaced independently, isolating costs and enabling flexible optimization.

The Self-Consistency mechanism remains the most computationally intensive component and the main latency bottleneck. Its pluggable structure allows iteration reduction or replacement,

offering an effective trade-off between efficiency and accuracy for latency-sensitive scenarios. Overall, SchemaRAG's non-monolithic architecture ensures that components can be individually optimized, updated, or extended, supporting both high performance and maintainability across diverse operational requirements.

4.6 SAR Evaluation (RQ.4)

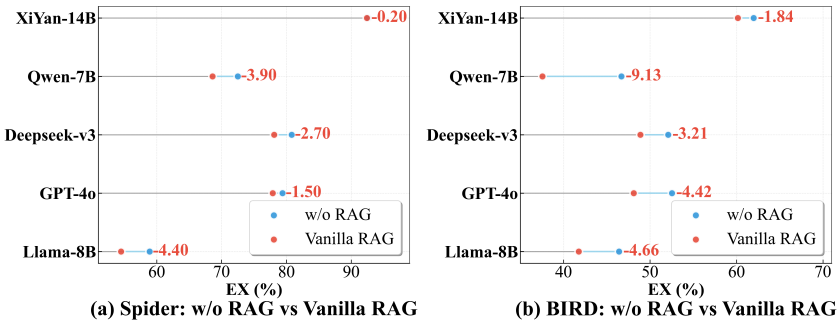


Fig. 11. Performance Comparison: SchemaRAG without RAG vs Vanilla RAG

4.6.1 Analysis of Vanilla RAG. To investigate the impact of vanilla RAG methods, we compared the performance of SchemaRAG without any RAG (baseline) against variants employing vanilla RAG techniques, such as text-based similarity retrieval and entity-masked retrieval. As illustrated in Figure 11, incorporating vanilla RAG often leads to counterproductive effects, where the EX decrease compared to the no-RAG baseline. This negative impact is particularly pronounced on smaller models like Qwen-7B and Llama-8B.

The counterproductive effect arises because vanilla RAG methods rely on superficial textual similarities, often retrieving examples that are semantically similar but structurally mismatched to the target SQL syntax (as shown in Figure 1). Smaller models, with limited reasoning capacity and context windows, are more susceptible to confusion from these irrelevant or misleading examples, leading to hallucinations and incorrect SQL generation. Larger models can better filter out noise due to their superior understanding and robustness, but even they do not benefit significantly from vanilla RAG without schema-aware enhancements. This underscores the necessity of our SAR module, which refines retrieval to focus on syntactic alignment, mitigating these issues and enabling consistent performance gains across model sizes.

4.6.2 Analysis of Embedding Visualization. Figure 12 illustrates that the final embeddings produced by SAR (c) exhibit a substantially more structured and discriminative feature space compared to the schema-aware embedding (a). Through contrastive learning, these representations are further refined, leading to markedly enhanced separability between distinct SQL structures. The final embeddings (c) demonstrate clear clustering of samples with similar SQL syntactic patterns while maintaining well-defined boundaries between dissimilar ones. We quantify this progressive improvement using the Silhouette Score, which evaluates clustering quality: it increases from 0.42 for the schema-aware embedding (a) to 0.78 for the final embeddings (c). This refined embedding space provides a robust foundation for retrieving high-quality few-shot examples, a capability explored in the following sections.

4.6.3 Hyperparameter Sensitivity Analysis. We conducted a sensitivity analysis to determine the optimal learning rate and contrastive temperature using a 3-layer, 4-head Transformer baseline.

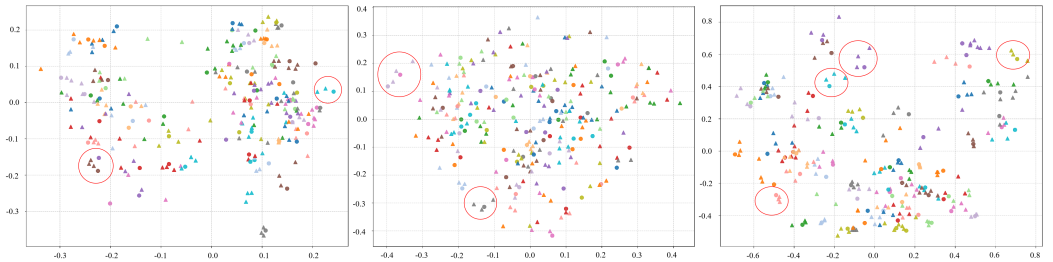
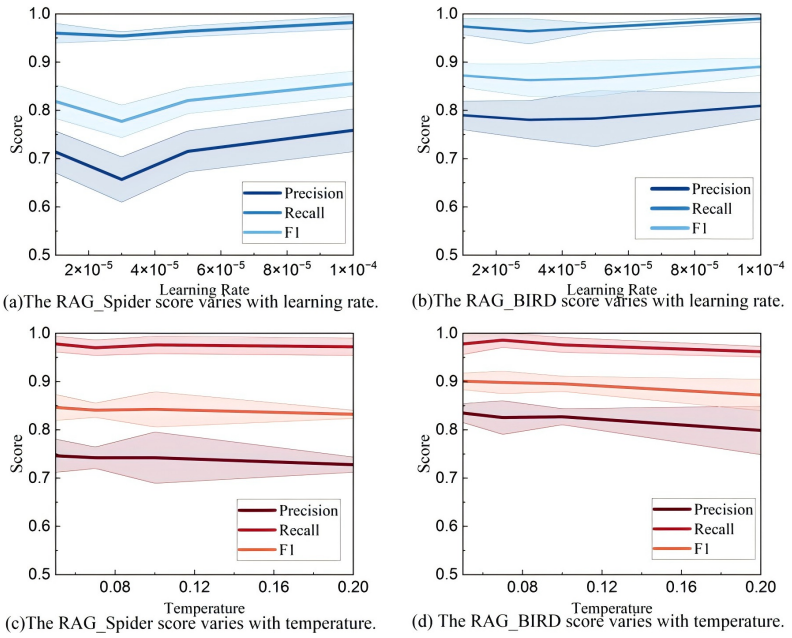

(a)2D Visualization of Schema aware Embeddings (b)2D Visualization of Question Embeddings (c)2D Visualization of Enhance Embeddings

Fig. 12. Visualization of different embedding spaces via Principal Component Analysis (PCA). Points of the same color represent pairs with similar SQL syntactic structures. Shapes distinguish between input queries (circles) and candidate examples from our RAG datasets (triangles).

As shown in Figure 13, a learning rate of 10^{-4} (with $T = 0.05$) achieved the best F1-scores on RAG_Spider (0.8554) and RAG_BIRD (0.8905). When fixing $LR = 10^{-4}$, a temperature of 0.05 further yielded the highest scores (0.8554 and 0.9006), indicating that a sharper contrastive objective enhances feature discrimination. Thus, we set $LR = 10^{-4}$ and $T = 0.05$ for all subsequent experiments.



(a)The RAG_Spider score varies with learning rate. (b)The RAG_BIRD score varies with learning rate.
(c)The RAG_Spider score varies with temperature. (d) The RAG_BIRD score varies with temperature.

Fig. 13. The performance of SAR with different temperature parameters and learning rates on the RAG_Spider and RAG_BIRD datasets, averaged over 5 runs with mean and variance reported.

4.6.4 Impact of Model Architecture. We then investigated the impact of model architecture, varying the number of Transformer layers and attention heads. Figure 14 reveals that the optimal architecture is dataset-dependent. For the more challenging RAG_BIRD dataset, a leaner architecture (2 layers,

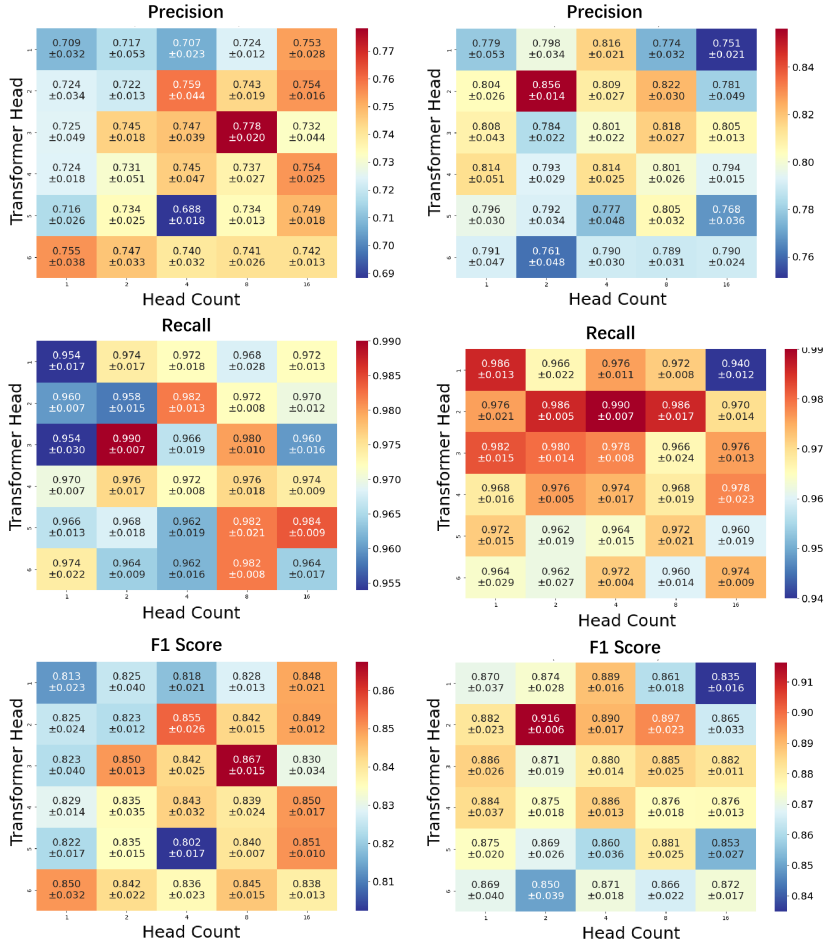


Fig. 14. The performance of SAR with different numbers of attention heads and Transformer layers on the RAG_Spider(left) and RAG_BIRD(right) datasets, averaged over 5 runs with mean and variance reported.

2 heads) performed best. Conversely, for RAG_Spider, a deeper model (3 layers, 8 heads) better captured finer-grained distinctions. This highlights the importance of tailoring model capacity to dataset complexity. Excessively deep or wide architectures led to performance degradation on both datasets, likely due to over-parameterization.

4.7 Ablation Experiments (RQ. 5)

To isolate and understand the contribution of each module within our framework, we conducted a comprehensive set of ablation experiments across five distinct datasets. We systematically removed three key components: the SchemaLinker, the SAR, and the POSG. The results, detailed in Tables 5 and 6, confirm that the competitive performance of SchemaRAG arises from the powerful synergy between its components. Additional experimental data can be found in Appendix G.

The ablation study indicates that SAR is the most critical component of our framework. As shown in Table 5, removing it causes the largest performance degradation across all models and metrics. For instance, the execution accuracy (EX) of GPT-4o on the BIRD dataset drops from 68.90%

Table 5. Ablation study of SchemaRAG, • indicates a result that is higher than the corresponding value of the SchemaRAG

Model	LLM	Spider		BIRD		Spider 2.0	Beaver	UniSQL
		EX	EM	EX	VES	EX	EX	EX
w/o SchemaLinker	GPT-4o	85.60	66.90•	65.19	30.80	15.49	7.05	46.78
	Deepseek-v3	85.30	78.10•	64.73	30.93	10.21	6.12	44.20
	Qwen-7B	78.10	53.10	55.22	28.22•	7.32	2.65	37.07
	XiYanSQL-14B	92.10	89.70•	63.30	31.32	18.84	10.46	47.37
	Llama-8B	77.70	60.50	55.35	28.95	6.84	2.12	38.18
w/o SAR	GPT-4o	79.40	26.70	52.54	23.89	13.45	6.12	40.63
	Deepseek-v3	80.80	20.20	52.09	22.00	9.02	5.40	39.04
	Qwen-7B	72.50	22.00	46.68	21.19	6.42	2.32	32.49
	XiYanSQL-14B	92.60	88.50	61.99	28.88	18.52	10.28	46.57
	Llama-8B	58.90	14.30	46.41	20.12	5.21	1.61	29.09
w/o POSG	GPT-4o	86.45	65.02	66.80	30.88	15.76	7.17	47.60
	Deepseek-v3	86.60	74.20	66.28	31.00	10.55	6.32	45.64
	Qwen-7B	79.50	53.20	54.12	26.23	7.37	2.65	37.23
	XiYanSQL-14B	92.60	86.50	65.04	33.63	19.14	10.63	48.11
	Llama-8B	79.80	62.24	59.26	29.82	7.17	2.22	39.99
SchemaRAG	GPT-4o	**85.60**	66.80	**68.90**	**31.98**	**15.90**	**7.18**	**48.05**
	Deepseek-v3	**89.60**	75.90	**68.38**	**31.05**	**10.79**	**6.67**	**46.88**
	Qwen-7B	**80.40**	**53.30**	**54.50**	25.10	**7.50**	**2.87**	**37.60**
	XiYanSQL-14B	**93.30**	88.00	**65.25**	**34.54**	**19.20**	**10.05**	**48.34**
	Llama-8B	**80.80**	**64.50**	**60.37**	**29.92**	**7.31**	**2.39**	**40.43**

Table 6. Loss/Win counts, Wilcoxon signed-rank test, and the Friedman test among models

Model	loss/win	p-value	F-rank
w/o SchemaLinker	129/31	9.40×10^{-3}	2.05
w/o SAR	159/1	3.81×10^{-6}	3.05
w/o POSG	158/2	9.40×10^{-3}	1.86
SchemaRAG	**34/446**	-	**1.15**

to 52.54%, and that of Llama-8B on the same dataset drops from 60.37% to 46.41%. On Spider, the EM score of Deepseek-v3 plunges from 75.90% to 20.20%, demonstrating that SAR is essential for generating syntactically correct SQL. This severe performance decline is consistently observed across the other datasets as well; for example, the EX scores for GPT-4o on Spider 2.0, Beaver, and UniSQL fall sharply by 2.45, 1.06, and 7.42 points, respectively. This trend is statistically validated in Table 6, where removing SAR results in the most significant performance drop (159 losses vs. 1 win) and the highest (worst) Friedman rank of 3.05. Moreover, the vanilla RAG using example retrieval exhibits virtually the same performance as removing SAR entirely, demonstrating that example retrieval alone offers little improvement without our schema-aware refinement mechanism.

Removing the SchemaLinker leads to a notable, though smaller, drop in performance across most models and datasets, confirming its importance in schema selection. For example, the EX score of XiYanSQL-14B on the BIRD dataset drops from 65.25% to 63.30%. This pattern of reduced

execution accuracy generally holds true for the more challenging datasets, including Spider 2.0, Beaver, and UniSQL. Interestingly, consistent EM score gains are observed on the Spider dataset for several models. This may result from the model having access to a broader schema context, which in simpler queries could occasionally allow it to match surface forms better, even if semantic precision suffers. Nevertheless, these gains are confined to a specific metric on a single dataset and do not outweigh the widespread drops in execution-based metrics across the board.

The POSG module, positioned as the final stage in the generation pipeline, primarily serves to select the optimal SQL query from multiple candidates. It functions more as a robustness enhancement mechanism for the final output rather than as a core performance-driving engine. This observation is clearly supported by our experimental results: across all five datasets, removing the POSG module leads to the smallest and least consistent drop in performance, with some metrics even showing slight improvements.(e.g., GPT-4o's EX on Spider increases from 85.60% to 86.45%).

Our analysis demonstrates that the complete SchemaRAG framework achieves competitive performance through the critical and complementary functions of its two core modules: SchemaLinker and SAR. The statistical significance of each component's contribution is confirmed by the Wilcoxon signed-rank test in Table 6, which shows that removing any module results in a significant performance degradation. These results underscore that the success of the framework is fundamentally driven by this effective integration, which addresses the distinct challenges of "what to use" (SchemaLinker) and "how to use it" (SAR) for generating complex and accurate SQL. The POSG module acts as a final refinement, further enhancing the overall robustness of the framework.

5 Limitations and Future Work

Despite SchemaRAG's strong benchmark results, its real-world applicability is still limited by scalability and robustness issues. Future work will focus on two directions: (1) improving SchemaLinker's scalability through hierarchical encoding and adaptive pruning for large databases, and (2) enhancing the exemplar retrieval mechanism to maintain embedding discriminability and generalization under large or unseen schemas. We will further validate these improvements on diverse Text-to-Structure benchmarks to make SchemaRAG more robust, scalable, and practical for real-world data analysis systems.

6 Conclusion

This paper proposes SchemaRAG, an innovative framework designed to address the challenge LLMs face in effectively understanding complex database schemas. Its efficacy stems from the synergy of three core components: the SchemaLinker, which accurately identifies relevant schema elements; the SAR, which retrieves syntactically similar examples to guide the generation process; and a Pareto-optimal selection mechanism, which ensures the robustness of the final output. Experimental results demonstrate the framework's superior performance. The "schema-aware" principle embodied in SchemaRAG not only offers a potent solution to this critical task but also provides valuable insights for enhancing the reasoning and generalization abilities of LLMs in broader Text-to-Structure applications.

7 Acknowledgments

This work is supported in part by the National Key Research and Development Program of China under Grant 2024YFF0908200, in part by the National Natural Science Foundation of China under Grants 62576289 and 62272078, and in part by the New Chongqing Youth Innovation Talent Project under Grant CSTB2024NSCQ-QCXMX003. Yi He was not supported by any of the above funding. Thanks to the High Performance Computing (HPC) clusters at Southwest University for providing computing resources.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774

[2] Jinze Bai, Shuai Bai, Yunfei Chu, et al. 2023. Qwen Technical Report. arXiv:2309.16609

[3] Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. In *NeurIPS*, Vol. 33. 1877–1901.

[4] Ruisheng Cao, Lu Chen, Zhi Chen, et al. 2021. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations. In *Proceedings of ACL-IJCNLP*. 2541–2555.

[5] Peter Baile Chen, Fabian Wenz, Yi Zhang, et al. 2024. BEAVER: An Enterprise Benchmark for Text-to-SQL. arXiv:2409.02038

[6] DeepSeek-AI, Aixin Liu, Bei Feng, et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437

[7] Xuemei Dong, Chao Zhang, Yuhang Ge, et al. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306

[8] Longxu Dou, Yan Gao, Mingyang Pan, et al. 2022. UniSAr: A Unified Structure-Aware Autoregressive Language Model for Text-to-SQL. arXiv:2203.07781

[9] Han Fu, Chang Liu, Bin Wu, et al. 2023. CatSQL: Towards Real World Natural Language to SQL Applications. *Proceedings of the VLDB Endowment* 16, 6 (Feb. 2023), 1534–1547.

[10] Yingqi Gao, Yifu Liu, Xiaoxia Li, et al. 2025. A Preview of XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL. arXiv:2411.08599

[11] Team GLM, Aohan Zeng, Bin Xu, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. arXiv:2406.12793

[12] Satya Krishna Gorti, Ilan Gofman, Zhaoyan Liu, et al. 2025. MSc-SQL: Multi-Sample Critiquing Small Language Models For Text-To-SQL Translation. In *Proceedings of NAACL(Long Papers)*, Vol. 1. 2145–2160.

[13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783

[14] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of ICML*, Vol. 80. 1856–1865.

[15] Binyuan Hui, Jian Yang, Zeyu Cui, et al. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186

[16] Nikhil Kandpal, Matthew Jagielski, Florian Tramèr, and Nicholas Carlini. 2023. Backdoor Attacks for In-Context Learning with Language Models. arXiv:2307.14692

[17] Fangyu Lei, Jixuan Chen, Yuxiao Ye, et al. [n. d.]. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows.

[18] Mike Lewis, Yinhan Liu, Naman Goyal, et al. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of ACL*. 7871–7880.

[19] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proceedings of the AAAI*, Vol. 37. 13067–13075.

[20] Haoyang Li, Jing Zhang, Hanbing Liu, et al. 2024. CodeS: Towards Building Open-source Language Models for Text-to-SQL. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 127.

[21] Jinyang Li, Binyuan Hui, Ge Qu, et al. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *NeurIPS*.

[22] Xing Han Lù. 2024. BM25S: Orders of magnitude faster lexical search via eager sparse scoring. arXiv:2407.03618

[23] Peixian Ma, Xialie Zhuang, Chengjin Xu, et al. 2025. SQL-R1: Training Natural Language to SQL Reasoning Model By Reinforcement Learning. arXiv:2504.08600

[24] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

[25] Charini Nanayakkara, Peter Christen, and Victor Christen. 2025. Unsupervised Evaluation of Entity Resolution. *JDIQ* 17, 1, Article 5 (2025), 31 pages.

[26] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *NeurIPS*, Vol. 36.

[27] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. In *Findings of EMNLP*. 8212–8220.

[28] Ge Qu, Jinyang Li, Bowen Li, et al. 2024. Before Generation, Align it! A Novel and Effective Strategy for Mitigating Hallucinations in Text-to-SQL Generation. In *Findings of ACL*. 5456–5471.

[29] Colin Raffel, Noam Shazeer, Adam Roberts, et al. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR* 21, 140 (2020), 1–67.

[30] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of EMNLP*. 9895–9901.

[31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347

[32] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300
[33] Zhili Shen, Pavlos Vougiouklis, Chenxin Diao, et al. 2024. Improving Retrieval-augmented Text-to-SQL with AST-based Ranking and Schema Pruning. In *Proceedings of EMNLP*. 7865–7879.
[34] Hugo Touvron, Louis Martin, Kevin Stone, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288
[35] Bailin Wang, Richard Shin, Xiaodong Liu, et al. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of ACL*. 7567–7578.
[36] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of NeurIPS*. Article 1800, 14 pages.
[37] An Yang, Baosong Yang, Beichen Zhang, et al. 2025. Qwen2.5 Technical Report. arXiv:2412.15115
[38] Pengcheng Yin and Graham Neubig. 2017. A Syntactic Neural Model for General-Purpose Code Generation. In *Proceedings of ACL(Long Papers)*. 440–450.
[39] Tao Yu, Rui Zhang, Kai Yang, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. 3911–3921.
[40] Chao Zhang, Yuren Mao, Yijiang Fan, et al. 2024. FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis. In *SIGMOD*. 93–105.
[41] Hanchong Zhang, Ruisheng Cao, Lu Chen, Hongshen Xu, and Kai Yu. 2023. ACT-SQL: In-Context Learning for Text-to-SQL with Automatically-Generated Chain-of-Thought. In *Findings of EMNLP*. 3501–3532.
[42] Kun Zhang, Xiexiong Lin, Yuanzhuo Wang, et al. 2023. ReFSQL: A Retrieval-Augmentation Framework for Text-to-SQL Generation. In *Findings of EMNLP*. 664–673.
[43] Yue Zhang, Yafu Li, Leyang Cui, et al. 2023. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. arXiv:2309.01219

Received July 2025; revised October 2025; accepted November 2025